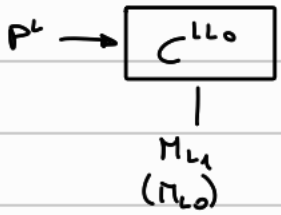
Es. 1 compilatore

Def: compilatore da L a Lo é un programma

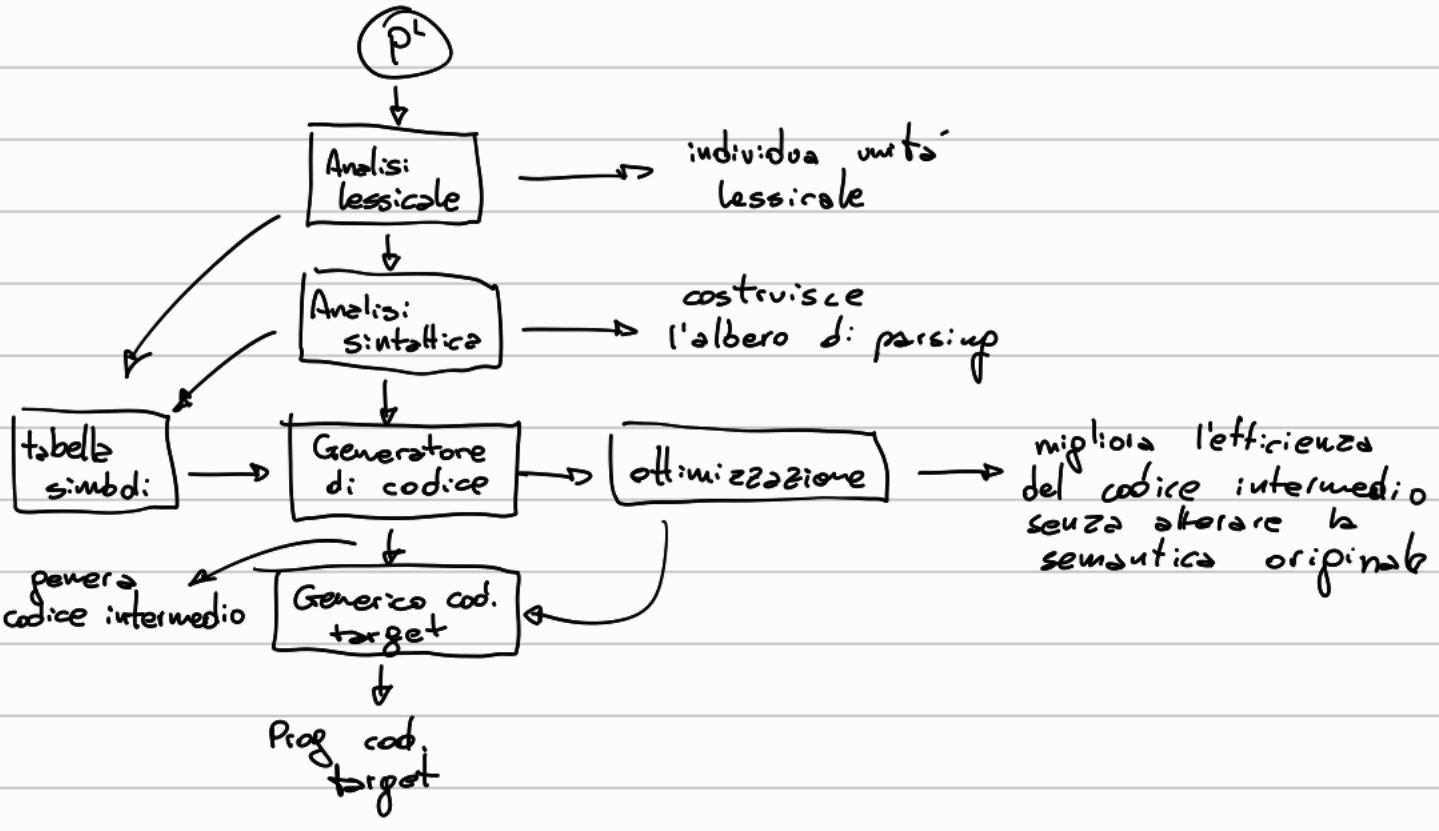
- C^{LLo} compilatore
- L linguaggio da implementare
- Lo linguaggio implementato
- $Prog^{L}$ programmi in L
- D insieme di dati manipolati da L

$$\forall P^{L} \in Prog^{L} \quad . \quad C^{LLo}(P^{L}) = P^{Lo} \text{ t.c. } \underbrace{\forall d \in D}_{\text{non decidibile}} . \; P^{L}(d) = P^{Lo}(d)$$



C^{LLo} puó essere scritto in L_1

La funzione del compilatore é quella di prendere un codice scritto in L e, prima di eseguirlo, lo traduce in Lo



Es. 2 induzione

$$\begin{cases} v(n) = 1 \\ v(e_1 + e_2) = v(e_1 - e_2) = v(e_1) + v(e_2) \end{cases}$$

$$\begin{cases} o(n) = 0 \\ o(e_1 + e_2) = o(e_1 - e_2) = o(e_1) + o(e_2) + 1 \end{cases}$$

$$v(e) = o(e) + 1$$

Caso base

$$v(e) = o(e) + 1 \xrightarrow{\text{def}} 1 = 0 + 1 \;;\; 1 = 1 \;\checkmark$$

Passo ind.

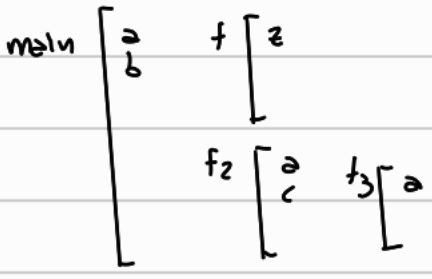
Itp. ind. : $v(e) = o(e) + 1$

$e = e_1 + e_2 \rightarrow$ analogo per $e_1 - e_2$

$$v(e) = v(e_1 + e_2) = v(e_1) + v(e_2) = \underbrace{o(e_1) + 1 + o(e_2)}_{\downarrow \text{def}\; o(e_1 + e_2)} + 1 = o(\overbrace{e_1 + e_2}^{e}) + 1 \overset{!}{=} o(e) + 1 \;\checkmark$$

Es. 3 scoping

```
{ int a=1; int b=2;
  void f(){ int z=a-b; }
  void f2(){
      int a=3; int c=4;
      void f3(){
          int a=c+b;
          f(); }
      b=a*b;
      f3(); }
  f2(); }
```



Profondità dichiarazioni

$Sd(main) = 0$

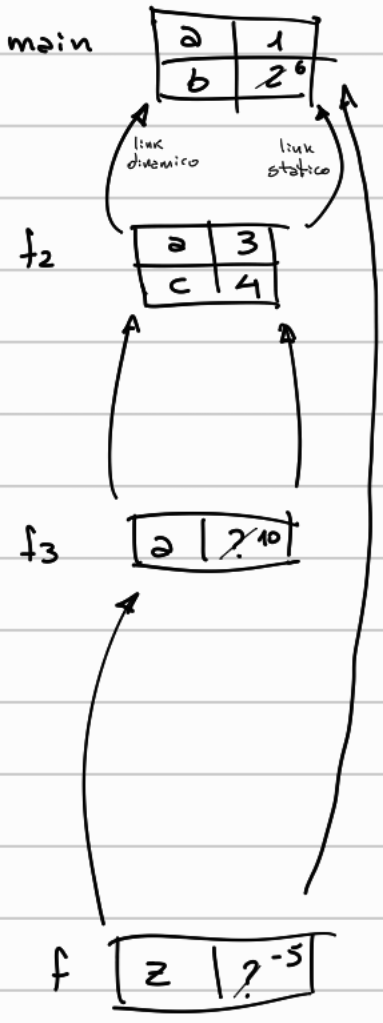
$Sd(f) = Sd(f2) = 1$

$Sd(f_3) = 2$

Seq. chiamate

main → f_2 → f_3 → f

Scoping statico



link statico $K_{f_2} = Sd(main) - Sd(f_2) + 1 = 0 - 1 + 1 = 0$

esecuzione f_2: b=a*b; $CS(f_2) = main$

- a locale
- b => $N_b = Sd(f_2) - Sd(main) = 1 - 0 = 1$ => b dal main

link statico $K_{f_3} = Sd(f_2) - Sd(f_3) + 1 = 1 - 2 + 1 = 0$

esecuzione f_3: a = c+b; $CS(f_3) = f_2$

- a locale
- c => $N_c = Sd(f_3) - Sd(f_2) = 2 - 1 = 1$ => c da f_2
- b => $N_b = Sd(f_3) - Sd(main) = 2 - 0 = 2$ => b dal main

link statico $K_f = Sd(f_3) - Sd(f) + 1 = 2 - 1 + 1 = 2$

esecuzione f: z = a-b; $CS(f) = main$

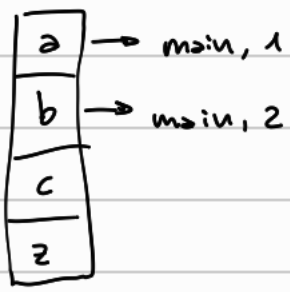
- z locale
- a => $N_a = Sd(f) - Sd(main) = 1 - 0 = 1$
- b => $N_b = Sd(f) - Sd(main) = 1 - 0 = 1$

} a e b dal main

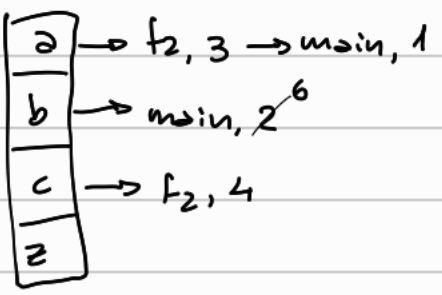
Tutte le chiamate terminano e lo stack deallocato

Scoping dinamico

main



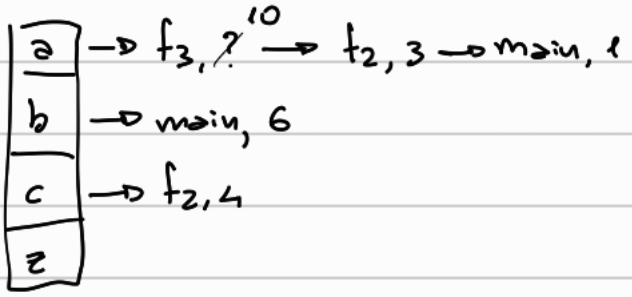
main → f_2



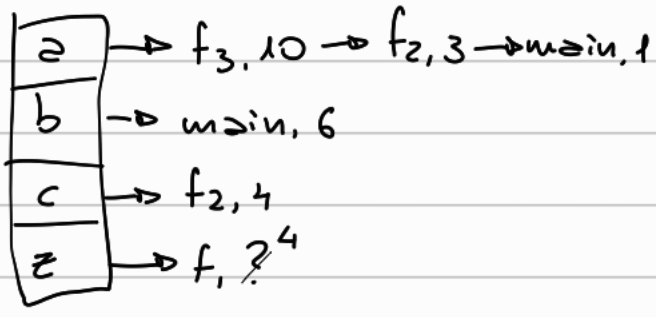
esecuzione f_2

b = a*b

$f_2 \to f_3$ esecuzione f_3



$f_3 \to f$ esecuzione f



Es. 4 scoping - binding

```
{ int a=2; int b=4;
  int p(int y){ return a+b-y; }
  int q(int f(int x)) {
        int a=3;
        return f(a)+b; }
  int exec(){
        int b=5;
        return q(p); }
  int z = exec(); }
```

← ① Scoping statico

← ③ Scoping dinamico + shallow binding

← ② Scoping dinamico + deep binding

- Scoping
  - statico → ambiente valido alla dichiarazione
  - dinamico
    - deep binding → creazione legame attuale e formale
    - shallow binding → chiama funzione parametro formale

① Scoping statico

z = exec() ⟶ q(p) ⟶ f(a)+b ⟶ a+b-y

b presa da main = 4 — y locale = 3

z = 7 ⟵ 7 ⟵ 3+4=7 ⟵ 2+4-3=3

② Scoping dinamico + deep binding

z = exec() ⟶ q(p) ⟶ f(a)+b ⟶ a+b-y

b presa da exec = 5 — y locale = 3

z = 9 ⟵ 9 ⟵ 4+5=9 ⟵ 2+5-3=4

③ Scoping dinamico + shallow binding

z = exec() ⟶ q(p) ⟶ f(a)+b ⟶ a+b-y

b preso da exec = 5 — y locale = 3

z = 10 ⟵ 10 ⟵ 5+5=10 ⟵ 3+5-3=5

Es. 5

```
void pippo (int a, int b){
    a = 5+a;
    b = 3b; }
void main (){
    int x=4; int y=5;
    pippo(x,y); }
```

Passaggio per valore

Il valore del parametro attuale viene passato come valore iniziale del corrispondente parametro formale.

main : x → [4], y → [5]

pippo : a → [4], b → [5] — copia

exe pippo : a → [9], b → [15]

terminazione pippo : main :

x → [4], y → [5] — valori non modificati

Passaggio per parametro

Viene passato un cammino di accesso al parametro formale

main : $x \to \boxed{4}$, $y \to \boxed{5}$

pippo : a , b

exe pippo : $x \to \boxed{8} \leftarrow a$, $y \to \boxed{15} \leftarrow b$

terminazione pippo : main : $x \to \boxed{8}$, $y \to \boxed{15}$ i parametri formali vengo cambiati

Es. 6 derivazione

$\rho = [x \mapsto l_x,\ y \mapsto 5]$ $\quad\quad$ $\Delta = [x \mapsto \text{:int var},\ y \mapsto \text{:int}]$

$\sigma = [l_x \mapsto 2]$

$x := 3x + y$

$$\frac{\rho \vdash \langle 3x+y, \sigma\rangle \to^* \langle k, \sigma\rangle}{\rho \vdash \langle x := 3x+y, \sigma\rangle \to \langle x := k, \sigma\rangle}$$

valutazione di x $\quad$ $\dfrac{\rho \vdash \langle x, \sigma\rangle \to \langle 2, \sigma\rangle}{\rho \vdash \langle 3x+y, \sigma\rangle \to \langle 3\cdot 2+y, \sigma\rangle}$ $\rho(x) = l_x,\ \sigma(l_x) = 2$

valutazione di y $\quad$ $\dfrac{\rho \vdash \langle y, \sigma\rangle \to \langle 5, \sigma\rangle}{\rho \vdash \langle 3\cdot 2+y, \sigma\rangle \to \langle 3\cdot 2+5, \sigma\rangle}$ $\rho(y) = 5$

$\rho \vdash \langle x := 3\cdot 2+5, \sigma\rangle \to \rho \vdash \langle x := 6+5, \sigma\rangle \to \rho \vdash \langle x := 11, \sigma\rangle$

$\rho(x) = l_x \to \sigma[l_x \mapsto 11]$